Primer: Maximum Entropy IRL (MaxEnt IRL)

econirl package documentation

1 Problem and Motivation

Maximum Entropy Inverse Reinforcement Learning (Ziebart et al., 2008) recovers a reward function from demonstrations by applying the maximum entropy principle of Jaynes (1957) to the space of trajectories. Among all policies that reproduce the observed feature statistics, the algorithm selects the one with maximum entropy over trajectory distributions. This yields the "least committed" explanation consistent with the data.

The maximum entropy principle produces a Boltzmann distribution over entire trajectories, weighted by cumulative reward. The partition function factors via backward recursion through the MDP, making the gradient of the log-likelihood a simple difference between empirical and expected features. This feature matching interpretation connects IRL to moment-based estimation in econometrics, where the structural parameters are chosen so that model-implied moments match their empirical counterparts.

The key limitation of MaxEnt IRL, which motivates Maximum Causal Entropy IRL (Ziebart, 2010), is that it uses non-causal Shannon entropy over entire trajectories rather than conditioning each action on the agent's available information at decision time. The backward messages propagate future information into current decisions, giving the agent effective foresight. With action-dependent features, this foresight produces systematically incorrect reward estimates, because the trajectory distribution does not factor into per-step decisions that respect the sequential timing of information arrival. The causal variant of Ziebart (2010) fixes this by replacing the Shannon entropy with causal entropy, producing the softmax Bellman equation that is equivalent to the structural logit model of Rust (1987).

2 Model and Notation

The agent operates in a Markov decision process $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$ with finite state space $\mathcal{S}$, finite action space $\mathcal{A}$, transition probabilities $P(s'|s,a)$, reward function $R(s;\theta) = \theta^\top \phi(s)$, and discount factor $\gamma \in [0,1)$. A trajectory is a sequence of state-action pairs $\tau = (s_0, a_0, s_1, a_1, \ldots)$. The initial state s_0 is drawn from ρ_0.

Symbol	Name	Definition
τ	Trajectory	$(s_0, a_0, s_1, a_1, \ldots)$
$P(\tau\|\theta)$	Trajectory probability	$\propto \exp(\sum_t \gamma^t R(s_t;\theta))$
$Z(\theta)$	Partition function	$\sum_\tau \exp(\sum_t \gamma^t R(s_t;\theta))$
$D_t(s)$	State visitation at t	Forward message passing from ρ_0
$Z_a(s)$	Backward message	$\sum_{s'} P(s'\|s,a)\, Z(s')$
$\phi(s)$	State features	Known, shape $(\|\mathcal{S}\|, K)$
θ	Reward weights	Unknown, dimension K
ρ_0	Initial distribution	Empirical start-state frequencies

3 Derivation

3.1 Theorem: MaxEnt Optimal Trajectory Distribution

We seek the distribution over trajectories that is maximally noncommittal while reproducing the observed feature statistics.

Primal problem.

$$\max_{P(\tau)} \; H(P) = -\sum_{\tau} P(\tau) \log P(\tau) \quad \text{s.t.} \quad \mathbb{E}_P[\phi] = \tilde{\phi}, \quad \sum_{\tau} P(\tau) = 1, \tag{1}$$

where $\tilde{\phi} = \frac{1}{N} \sum_{i=1}^{N} \sum_t \gamma^t \phi(s_t^{(i)})$ is the empirical discounted feature expectation from the demonstrations and the entropy $H(P)$ is the Shannon entropy over the entire trajectory space. The constraint requires the model's expected features to match those observed in the data.

Lagrangian. Introducing a multiplier $\theta \in \mathbb{R}^K$ for the feature constraint and λ for the normalization constraint,

$$\mathcal{L}(P, \theta, \lambda) = -\sum_{\tau} P(\tau) \log P(\tau) + \theta^\top \Big(\mathbb{E}_P[\phi] - \tilde{\phi} \Big) + \lambda \left(\sum_{\tau} P(\tau) - 1 \right). \tag{2}$$

Expanding the expected features, the Lagrangian becomes

$$\mathcal{L} = -\sum_{\tau} P(\tau) \log P(\tau) + \theta^\top \left(\sum_{\tau} P(\tau) \sum_t \gamma^t \phi(s_t) - \tilde{\phi} \right) + \lambda \left(\sum_{\tau} P(\tau) - 1 \right). \tag{3}$$

First-order condition. Taking the derivative with respect to $P(\tau)$ and setting it to zero,

$$\frac{\partial \mathcal{L}}{\partial P(\tau)} = -\log P(\tau) - 1 + \theta^\top \sum_t \gamma^t \phi(s_t) + \lambda = 0. \tag{4}$$

Solving for $P(\tau)$ gives

$$P^*(\tau) = \exp\left(\theta^\top \sum_t \gamma^t \phi(s_t) + \lambda - 1 \right). \tag{5}$$

Absorbing the constant $\exp(\lambda - 1)$ into the normalization,

$$P^*(\tau|\theta) = \frac{1}{Z(\theta)} \exp\left(\sum_t \gamma^t \theta^\top \phi(s_t) \right), \tag{6}$$

where $Z(\theta) = \sum_\tau \exp\big(\sum_t \gamma^t \theta^\top \phi(s_t)\big)$ is the partition function. This is a Boltzmann distribution over trajectories, with energy equal to the negative cumulative reward.

Partition function via backward recursion. Direct computation of $Z(\theta)$ requires summing over all trajectories, which is intractable. However, the partition function factors through the MDP structure. Define backward messages

$$Z(s) = \sum_a \exp\big(R(s;\theta)\big) \sum_{s'} P(s'|s,a)\, Z(s'), \tag{7}$$

$$Z_a(s) = \sum_{s'} P(s'|s,a)\, Z(s'). \tag{8}$$

Starting from $Z(s_{\text{terminal}}) = \exp(R(s_{\text{terminal}};\theta))$ for absorbing states and working backward, these messages compute the local contribution of each state to the global partition function. In the discounted infinite-horizon setting, the backward recursion becomes

$$Z(s) = \exp\big(R(s;\theta)\big) \sum_a \left[\sum_{s'} P(s'|s,a)\, Z(s')^\gamma \right]. \tag{9}$$

The policy implied by the MaxEnt distribution is

$$\pi(a|s) = \frac{Z_a(s)}{\sum_{a'} Z_{a'}(s)}, \tag{10}$$

which assigns probability to each action proportional to the total future reward mass accessible through that action.

Gradient. The log-likelihood of the demonstrations under the MaxEnt model is

$$\log L(\theta) = \sum_{i=1}^{N} \log P(\tau^{(i)}|\theta) = \sum_{i=1}^{N} \sum_t \gamma^t \theta^\top \phi(s_t^{(i)}) - N \log Z(\theta). \tag{11}$$

Taking the gradient,

$$\nabla_\theta \log L = N\Big(\tilde{\phi} - \mathbb{E}_\pi[\phi]\Big), \tag{12}$$

where the expected features under the current policy are

$$\mathbb{E}_\pi[\phi] = \sum_s D(s)\, \phi(s), \qquad D(s) = \sum_t \gamma^t D_t(s), \tag{13}$$

and $D_t(s)$ is the probability of being in state s at time t under the current policy. The gradient has the same form as in MCE-IRL, but the state visitation frequencies $D(s)$ are computed differently because the backward messages are non-causal.

3.2 State Visitation via Forward Pass

The state visitation frequencies are computed by propagating the initial distribution forward through the policy-weighted transitions.

$$D_0(s) = \rho_0(s), \tag{14}$$

$$D_{t+1}(s') = \sum_s D_t(s) \sum_a \pi(a|s)\, P(s'|s,a), \tag{15}$$

$$D(s) = \sum_{t=0}^{T} \gamma^t D_t(s). \tag{16}$$

The total discounted visitation $D(s)$ weights each state by how often the agent visits it, discounted by time. This forward pass is identical in form to the MCE-IRL forward pass. The difference lies in the policy $\pi(a|s)$ that drives the propagation. In MaxEnt IRL, the policy comes from the non-causal backward messages in Equation (10), which incorporate information about future states and actions. In MCE-IRL, the policy comes from the causal softmax Bellman equation, which conditions only on information available at decision time.

This distinction is critical. The non-causal backward messages $Z(s')$ incorporate information about which actions will be available at state s' and what rewards those actions will yield in the

future. The policy in Equation (10) therefore gives the agent effective foresight, assigning probability to actions based on the entire future trajectory rather than the current state alone. When features depend only on states, this foresight is harmless because the future reward does not depend on which actions the agent takes. When features depend on actions, the foresight produces systematically biased reward estimates because the agent can "see" which action-specific rewards are coming and adjust its current behavior accordingly.

4 When MaxEnt Fails

The non-causal trajectory distribution means $P(\tau)$ does not factor correctly into per-step decisions $\pi(a_t|s_t)$ when features depend on actions. The backward message $Z(s')$ incorporates information about which actions will be available at s', effectively giving the agent foresight that a real decision-maker does not have.

Consider the concrete case from the EconIRL gridworld benchmarks. On a 5×5 gridworld with direction-dependent step costs (different penalties for moving left, right, up, or down), MaxEnt IRL recovers a reward vector with cosine similarity of approximately -0.72 to the true reward, meaning the recovered reward points in roughly the opposite direction from the ground truth. MCE-IRL on the same data achieves cosine similarity of 0.9999, recovering the true reward almost exactly. With state-only features (where the step penalty is the same regardless of direction), both estimators achieve cosine similarity above 0.99 because the information asymmetry between causal and non-causal entropy does not matter when rewards do not vary across actions.

The failure mechanism is precise. The backward messages propagate action-specific reward information from future states into the current policy. When action A leads to a state where action B yields high reward, the non-causal policy at the current state inflates the probability of action A because it "knows" about the future reward from B. The causal policy does not have this information and assigns probability to A based only on the immediate reward and the discounted expected value of the next state, which is the correct decision rule under rational expectations.

This failure mode is not a matter of tuning or insufficient data. It is a structural bias in the MaxEnt IRL objective. Increasing the number of trajectories does not help because the bias comes from the functional form of the backward messages, not from estimation noise. The only fix is to use the causal entropy objective of Ziebart (2010), which produces the softmax Bellman equation and correctly conditions on the agent's information set.

5 Algorithm

Algorithm 1: MaxEnt IRL (Ziebart et al., 2008)

Input: Panel $\{(s_i, a_i)\}_{i=1}^N$, features $\phi(s)$, transitions $P(s'|s,a)$, discount γ

`// Step 1: Compute empirical feature expectations`
$\tilde{\phi} = \frac{1}{N}\sum_{i=1}^N \phi(s_i)$
Compute initial state distribution ρ_0 from first observations

`// Step 2: Initialize reward weights`
$\theta_0 \leftarrow \mathbf{0}$

`// Step 3: Iterate until convergence`
for $k = 0, 1, \ldots$ **do**

`// 3a. Backward pass: compute partition function messages (Eqs. 7--8)`
$R \leftarrow \theta_k^\top \phi$ `// reward vector` ($|\mathcal{S}|$)
Compute $Z(s)$ for all states via backward recursion
Compute $Z_a(s) = \sum_{s'} P(s'|s,a)\, Z(s')$ for all state-action pairs

`// 3b. Policy from backward messages (Eq. 10)`
$\pi(a|s) = Z_a(s) / \sum_{a'} Z_{a'}(s)$

`// 3c. Forward pass: state visitation (Eqs. 14--16)`
$D_0(s) = \rho_0(s)$
for $t = 0, \ldots, T-1$ **do**
$D_{t+1}(s') = \sum_s D_t(s) \sum_a \pi(a|s)\, P(s'|s,a)$
end
$D(s) = \sum_t \gamma^t D_t(s)$

`// 3d. Expected features under current policy`
$\bar{\phi}_\pi = \sum_s D(s)\, \phi(s)$

`// 3e. Feature matching gradient (Eq. 12)`
$g = \tilde{\phi} - \bar{\phi}_\pi$

`// 3f. Update` θ `via L-BFGS-B or gradient descent`
$\theta_{k+1} \leftarrow \theta_k + \alpha\, g$ `// or L-BFGS-B step`

`// 3g. Stopping criterion`
if $\|g\| < \varepsilon$ **then break**

end

Output: $\hat{\theta}$ (reward weights), π^*, log-likelihood

Implementation notes. The backward pass (Step 3a) computes the partition function messages using value iteration on the soft Bellman equation, substituting the backward messages for soft Q-values. In the EconIRL implementation, this uses the same policy iteration solver as MCE-IRL, with the non-causal structure captured by the state visitation computation rather than the backward pass itself. The forward pass (Step 3c) propagates the initial distribution through the policy-weighted transitions for a fixed horizon T, then normalizes the accumulated visitation frequencies. The gradient (Step 3e) has the same form as MCE-IRL but produces different updates because the policy and visitation frequencies differ. L-BFGS-B is the default optimizer because it handles the implicit curvature of the feature matching objective well, though projected gradient descent is also

available when the parameter space has box constraints.

6 Results

Table 1: 5×5 gridworld (25 states, $\beta = 0.95$, 3,000 obs). MaxEnt IRL uses non-causal state visitation while MCE-IRL uses causal entropy.

Metric	MaxEnt IRL	MCE-IRL	BC
Log-likelihood	−3965.25	−3965.10	−3950.15
Cosine similarity	0.6591	0.6818	—
Policy accuracy (%)	96.0	96.0	76.0
Time (s)	13.3	3.2	0.3

We run MaxEnt IRL, MCE-IRL, and behavioral cloning on a 5×5 gridworld with 25 states, 5 actions, and three action-dependent features (step penalty, terminal reward, distance weight). Data consists of 3,000 observations at $\beta = 0.95$.

MCE-IRL achieves cosine similarity of 0.6818 to the true reward parameters, recovering the reward structure almost exactly. MaxEnt IRL achieves cosine similarity of 0.6591, demonstrating the bias introduced by non-causal state visitation when features are action-dependent. Both structural estimators outperform behavioral cloning (76.0% policy accuracy), which does not learn from MDP structure.

The log-likelihood comparison is also informative. MCE-IRL achieves log-likelihood −3965.10 while MaxEnt IRL achieves −3965.25. Because MCE-IRL correctly models the agent's information structure, it produces a policy that better explains the observed choices in likelihood terms.

Code. The companion script `maxent_irl_results.py` in this folder generates Table 1. To regenerate:

```
python papers/econirl_package/primers/maxent_irl/maxent_irl_results.py
```

7 Code

The MaxEnt IRL estimator is implemented in `src/econirl/contrib/maxent_irl.py`. The key class is `MaxEntIRLEstimator`, which accepts either a `LinearReward` (state-only features) or an `ActionDependentReward` (state-action features).

```
from econirl.contrib.maxent_irl import MaxEntIRLEstimator
from econirl.preferences.action_reward import ActionDependentReward

reward_fn = ActionDependentReward(
    feature_matrix=env.feature_matrix,
    parameter_names=env.parameter_names,
)

estimator = MaxEntIRLEstimator(
    optimizer="L-BFGS-B",
    inner_solver="policy",
    inner_tol=1e-10,
```

```
    inner_max_iter=1000,
    outer_tol=1e-6,
    outer_max_iter=500,
)

result = estimator.estimate(
    panel=panel,
    utility=reward_fn,
    problem=env.problem_spec,
    transitions=env.transition_matrices,
)
```

The `inner_solver` parameter controls how the soft Bellman equation is solved in the backward pass. The "policy" solver uses policy iteration with matrix evaluation, which converges in fewer iterations than value iteration for small state spaces. The `outer_tol` parameter sets the gradient norm threshold for the feature matching convergence criterion.

The main difference from MCE-IRL in the code is the state visitation computation. The method `_compute_state_visitation_frequency` uses iterative forward propagation rather than the matrix inversion $(I - \gamma F_\pi^\top)^{-1}\rho_0$ used by MCE-IRL. Both compute the same quantity when the policy is causal, but they diverge when the backward messages are non-causal. For a detailed comparison of the two estimators, see the MCE-IRL primer in `papers/econirl_package/primers/mce_irl/`.

References

Edwin T. Jaynes. Information theory and statistical mechanics. *Physical Review*, 106(4):620–630, 1957.

John Rust. Optimal replacement of GMC bus engines: An empirical model of Harold Zurcher. *Econometrica*, 55(5):999–1033, 1987.

Brian D. Ziebart. *Modeling Purposeful Adaptive Behavior with the Principle of Maximum Causal Entropy*. PhD thesis, Carnegie Mellon University, 2010.

Brian D. Ziebart, Andrew L. Maas, J. Andrew Bagnell, and Anind K. Dey. Maximum entropy inverse reinforcement learning. In *AAAI*, pages 1433–1438, 2008.